

Configuración, Secretos, Feature Flags y Entrega Progresiva

Resumen de dos sesiones del curso — Sistemas Distribuidos, Corte 2

Juan Esteban Oliveros Duran · pms-properties · jeoliveros-2023a@corhuila.edu.co

Sesión 1 — Configuration, secrets and feature flags

Idea central: la configuración, los secretos y las feature flags son la forma en que un sistema cambia de comportamiento sin recompilar ni redespargar código. Tratarlos con disciplina es lo que separa un despliegue aburrido de un incidente a las 2 a.m.

Configuración estricta al estilo 12-factor

- Toda la configuración que cambia entre entornos (URLs, flags, límites, credenciales) vive **fuera del código**: variables de entorno o un servicio de configuración, nunca hardcodeada ni en un archivo versionado con valores reales.
- El mismo artefacto (el mismo .jar, la misma imagen) se despliega en Local, Dev, Staging y Producción; solo cambia la configuración externa que recibe al iniciar.
- Un cambio de configuración no debería requerir una nueva compilación — si hace falta recompilar para cambiar un timeout, esa es una señal de diseño a corregir.

Secretos: nunca en git, con rotación

- Un secreto (contraseña de base de datos, clave de API, clave de firma de JWT) **jamás** se escribe en un repositorio — ni en el código, ni en un archivo de configuración versionado, ni siquiera en el historial de commits de una rama borrada.
- Flujo recomendado: el secreto vive en un *store* dedicado (gestor de secretos / vault) → se inyecta como variable de entorno al arrancar el servicio → el servicio lo lee del entorno, nunca de un archivo propio.
- Rotación: un secreto debe poder cambiarse sin tocar código y sin tumbar el servicio — rotarlo periódicamente y de inmediato si se filtra, en vez de darlo por 'seguro' hasta el fin del proyecto.

Validación fail-fast al arrancar

- Un servicio debe **fallar al iniciar** (fail fast) si falta una variable de configuración obligatoria o un secreto requerido — no arrancar 'a medias' y fallar más tarde, en producción, frente al primer request real que necesite ese valor.
- Esta validación temprana convierte un error de despliegue en algo visible de inmediato en los logs de arranque, en vez de un bug intermitente difícil de reproducir.

Feature flags: deploy ≠ release

- Una *feature flag* separa **desplegar código** (llevarlo a producción, apagado) de **liberar una funcionalidad** (encenderla para los usuarios) — son dos decisiones distintas que ya no tienen que ocurrir en el mismo momento.
- Esto permite un *canary rollout*: encender la funcionalidad para un pequeño porcentaje de usuarios o un entorno controlado primero, medir, y solo después ampliar al 100%.

- Si algo sale mal, el **rollback es instantáneo**: apagar la flag, sin necesidad de un nuevo despliegue ni de revertir un commit.

Escenario de advertencia — la fuga de secretos a las 2 a.m.

Alguien sube, por error, un archivo `.env` con credenciales reales en un commit normal. El repositorio es público o el commit llega a una rama compartida; horas después, algo empieza a fallar — tráfico extraño, un servicio externo bloquea las llamadas. **Causa raíz**: el secreto quedó expuesto en el historial de git, visible para cualquiera con acceso al repo, incluso después de borrar el archivo en un commit posterior (el historial lo conserva). **Solución**: rotar el secreto comprometido de inmediato (invalidarlo y generar uno nuevo), nunca confiar en 'borrarlo después' como mitigación; limpiar el historial de git si es estrictamente necesario; y, a futuro, usar un gestor de secretos + `.gitignore` + revisión en el PR para que esto no pueda volver a subirse.

Errores comunes señalados en la sesión

- Guardar un secreto de 'ejemplo' en un archivo de configuración versionado, pensando que no es el real.
- Dejar que un servicio arranque sin validar su configuración y descubrir la falta de una variable solo cuando un usuario real la necesita.
- Usar una variable de entorno con un valor distinto en cada entorno para decisiones que en realidad deberían ser una feature flag controlable en caliente.

Sesión 2 — Planning: secure config and progressive delivery

Idea central: no basta con conocer los principios de la Sesión 1 — un equipo necesita un **plan concreto** para sus propios secretos y flags, una estrategia de liberación progresiva, y la disciplina de dividir el trabajo de 'endurecimiento' (hardening) en historias que de verdad se puedan probar.

Un plan de secretos real (no solo el principio)

- Para cada secreto del sistema se define, por escrito: **dueño** (quién es responsable de él), **store** (dónde vive: el gestor de secretos elegido), **mecanismo de inyección** (cómo llega al servicio — variable de entorno al arrancar), **principio de mínimo privilegio** (qué servicio puede leer qué secreto, y nada más) y **política de rotación** (cada cuánto, y qué hacer ante una fuga).
- Este plan se escribe *antes* de implementar, igual que un contrato de API — es la aplicación del mismo principio de 'contrato primero' a la configuración sensible.

Estrategia de feature flags como artefacto de equipo

- Cada flag tiene, documentado: un **nombre** consistente y buscable, un **dueño**, una **fecha de remoción** prevista (una flag no es para quedarse para siempre — es deuda si nadie la retira), y un **valor por defecto apagado (off)** hasta que se decida encenderla.
- Sin esta disciplina, el código se llena de flags huérfanas que nadie recuerda para qué servían, y cada una es un camino de código adicional que hay que seguir probando indefinidamente.

Entrega progresiva: canary + rollback

- Una liberación progresiva típica avanza en etapas controladas — por ejemplo **5%** → **50%** → **100%** del tráfico o de los usuarios — observando métricas y errores en cada etapa antes de avanzar a la siguiente.
- La posibilidad de **rollback instantáneo** (apagar la flag) es lo que hace seguro experimentar con cada etapa: si el 5% muestra un problema, se apaga sin necesidad de revertir un despliegue.

Dividir historias de 'hardening' en algo probable

- El trabajo de endurecer la configuración y los secretos de un MVP 2 se divide en historias pequeñas con criterios de aceptación verificables (ej.: 'el servicio falla al arrancar si falta la variable X', 'la flag Y tiene fecha de remoción documentada'), en vez de una tarea genérica 'mejorar la seguridad de la configuración' que nadie puede marcar como terminada con certeza.

Escenario de advertencia — el release todo-o-nada

Un equipo despliega una funcionalidad grande directamente al 100% de los usuarios, sin flag y sin etapas, confiando en que 'ya se probó en staging'. Al llegar a producción aparece un caso de borde no previsto y afecta a todos los usuarios a la vez. **Causa raíz:** no existía ningún mecanismo para exponer el cambio solo a una fracción del tráfico, así que no hubo forma de contener el impacto ni de revertir sin un nuevo despliegue urgente. **Solución:** el cambio debía ir detrás de una feature flag con rollout progresivo (canary) y, ante la primera señal de error, apagarla — convirtiendo un incidente de producción en un ajuste de unos segundos.

Cómo se conecta con el trabajo del proyecto

Estas dos sesiones son la base directa de las decisiones de configuración, secretos y despliegue que el proyecto **pms-property** debe declarar explícitamente antes de implementar cualquier servicio: qué va en variables de entorno, cómo se inyectan los secretos de cada microservicio (*booking, payment, identity*,

notification, catalog), y qué cambios arquitectónicos grandes (como el paso de Saga coreografiada a orquestada) deberían poder liberarse de forma progresiva en vez de todo-o-nada.

Nota: este resumen excluye intencionalmente las secciones de autoevaluación ("Self-check") al final de cada sesión, por solicitud expresa.